

Capstone Project

AI-Powered News Insight System

Using Machine Learning, Transformers & Generative AI (RAG)

Project Objective

Build an end-to-end AI system that can:

- Classify news articles using basic Machine Learning algorithms
- Improve understanding using a Transformer-based Deep Learning model
- Answer user questions using Generative AI with RAG (Retrieval-Augmented Generation)

Dataset columns used:

- text — News article content
- category — Business, Tech, Politics, Sports, Entertainment

Phase 1: Data Understanding & Preprocessing

Step 1: Load the Dataset

- Load the CSV file from the provided Google Drive link
- Check shape, column names, and missing values

```
bbc_news_og_ds = pd.read_csv("BBC_Capstone_Dataset.csv")
```

```
bbc_news = bbc_news_og_ds.copy()
```

```
bbc_news.head()
```

	category	text
0	tech	tv future in the hands of viewers with home th...
1	business	worldcom boss left books alone former worldc...
2	sport	tigers wary of farrell gamble leicester say ...
3	sport	yeading face newcastle in fa cup premiership s...
4	entertainment	ocean s twelve raids box office ocean s twelve...

[+ Code](#)[+ Markdown](#)

```
bbc_news.shape
```

```
(2225, 2)
```

Step 2: Text Cleaning

Perform basic text preprocessing:

- Convert text to lowercase
- Remove punctuation and numbers
- Remove stopwords

```
# Removing punctuations,numbers and stopwords
bbc_news['text']=bbc_news['text'].str.lower()
stop_words = set(stopwords.words('english'))
def clean_text(text):
    tokens = word_tokenize(text)
    clean_text = ' '.join([word for word in tokens if word.isalpha() and word not in stop_words])
    return clean_text
```

```
bbc_news['processed_text']=bbc_news['text'].apply(clean_text)
bbc_news = bbc_news.drop('text',axis=1)
```

Step 3: Label Encoding

Convert news categories into numerical labels. Example mapping:

- Business → 0
- Tech → 1
- Politics → 2
- Sports → 3
- Entertainment → 4

```
le = LabelEncoder()  
bbc_news['encoded_category'] = le.fit_transform(bbc_news['category'])  
  
bbc_news = bbc_news.drop('category', axis=1)
```

Phase 2: Machine Learning

Step 4: Feature Extraction

- Splitting dataset into training and testing dataset.

```
#Splitting dataset into training(90%) and testing(10%)  
X_train,X_test,y_train,y_test = train_test_split(bbc_news['processed_text'],bbc_news['encoded_category'],random_state=42,test_size=0.1)
```

- Convert text into numerical features using TF-IDF Vectorizer

```
tf_idf_vec = TfidfVectorizer()  
  
X_train_tfidf = tf_idf_vec.fit_transform(X_train)  
X_test_tfidf = tf_idf_vec.transform(X_test)
```

Step 5: Train ML Model

- Train a Logistic Regression classifier on the TF-IDF features

```
logistic_reg = LogisticRegression()
```

```
logistic_reg.fit(X_train_tfidf,y_train)
```

▼ LogisticRegression ⓘ ⓘ
LogisticRegression()

```
y_pred = logistic_reg.predict(X_test_tfidf)
```

Step 6: Model Evaluation

Evaluate model performance using:

- **Accuracy score**

```
print("Accuracy_score",accuracy_score(y_test, y_pred))
```

Accuracy_score 0.9596412556053812

- **Confusion matrix**

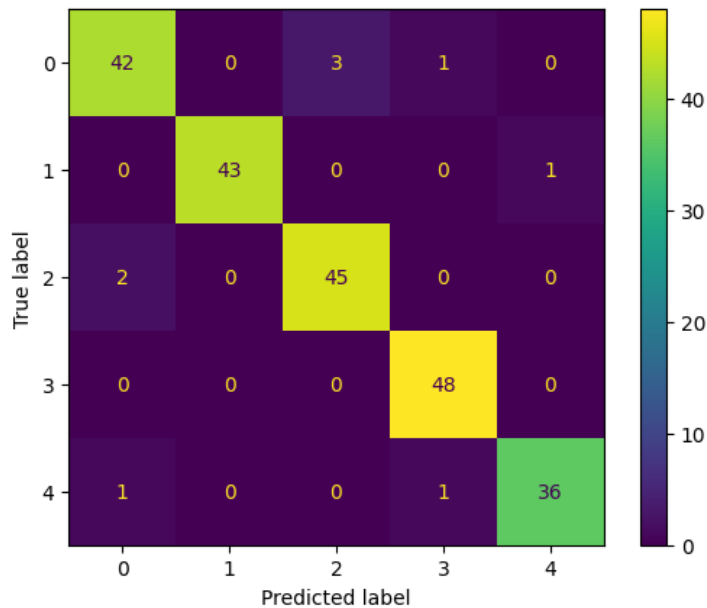
```
cm = confusion_matrix(y_test,y_pred)  
print("Confusion Matrix \n",cm)
```

Confusion Matrix

```
[[42  0  3  1  0]  
 [ 0 43  0  0  1]  
 [ 2  0 45  0  0]  
 [ 0  0  0 48  0]  
 [ 1  0  0  1 36]]
```

```
# Creating a visually appealing confusion matrix
#Category mappings ==> 'business': 0, 'entertainment': 1, 'politics': 2, 'sport': 3, 'tech': 4
plt.figure(figsize=(8, 8))
cm_display = ConfusionMatrixDisplay(confusion_matrix=cm,
                                     display_labels=logistic_reg.classes_)
cm_display.plot()
plt.show()
```

<Figure size 800x800 with 0 Axes>



- **Classification report**

```
print("Classification Report \n", classification_report(y_test, y_pred))
```

```
Classification Report
              precision    recall  f1-score   support

     0:       0.93       0.91       0.92         46
     1:       1.00       0.98       0.99         44
     2:       0.94       0.96       0.95         47
     3:       0.96       1.00       0.98         48
     4:       0.97       0.95       0.96         38

 accuracy          0.96
 macro avg         0.96
weighted avg         0.96
```

Phase 3: Deep Learning – Transformer Model

Step 7: Load a Pre-trained Transformer Model

- Use a lightweight pre-trained model: distilbert-base-uncased

```

#Creating a copy of pre-processed dataset and renaming columns as transformers must required 'label' as a target column name.
bbc_news_trans_cp = bbc_news.copy()
bbc_news_trans_cp.columns = ['text', 'label']

#Converting the dataframe to HuggingFace Dataset
bbc_news_trans_ds = Dataset.from_pandas(bbc_news_trans_cp)

#Splitting dataset into training and testing
bbc_news_trans_ds_split = bbc_news_trans_ds.train_test_split(test_size=0.1, seed=42)

#bbc_news_trans_ds_split

## Loading a pre-trained Transformer model

pre_trained_model = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(pre_trained_model)
model = AutoModelForSequenceClassification.from_pretrained(
    pre_trained_model,
    num_labels=5
)

```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

config.json: 100%  483/483 [00:00<00:00, 46.6kB/s]

tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 5.56kB/s]

vocab.txt:  232k/? [00:00<00:00, 5.96MB/s]

tokenizer.json:  466k/? [00:00<00:00, 10.1MB/s]

model.safetensors: 100%  268M/268M [00:02<00:00, 200MB/s]

Loading weights: 100%  100/100 [00:00<00:00, 692.03it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weight]

Step 8: Text Processing Using Pre-built Transformer Pipeline

- Tokenizing data

```

def tokenize_data(sample_data):
    return tokenizer(sample_data["text"], padding=True, truncation=True, max_length=256)

tokenized_train_dataset = bbc_news_trans_ds_split['train'].map(tokenize_data, batched=True)
tokenized_test_dataset = bbc_news_trans_ds_split['test'].map(tokenize_data, batched=True)

tokenized_train_dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "label"])
tokenized_test_dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "label"])

```

Map: 100%  2002/2002 [00:01<00:00, 1799.80 examples/s]

Map: 100%  223/223 [00:00<00:00, 1683.32 examples/s]

- Defining a custom evaluation metric function to calculate the accuracy of the model.

```

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return {"accuracy": accuracy_score(labels, preds)}

```

- Setting the Training Arguments- We are training the model for 3 epochs with a batch size of 16.

```
training_args = TrainingArguments(
    output_dir="./news_distilbert_classification_model",
    num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="accuracy",
    logging_steps=20
)
```

- Train the model.

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_train_dataset,
    eval_dataset=tokenized_test_dataset, #using test dataset as evaluation dataset
    processing_class=tokenizer,
    compute_metrics=compute_metrics
)
```

```
print("\nTraining Model ...")
trainer.train()
```

Training Model ...

/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Was return super().apply(*args, **kwargs) # type: ignore[misc]

[189/189 01:18, Epoch 3/3]

Epoch	Training Loss	Validation Loss	Accuracy
1	0.360336	0.282870	0.964126
2	0.053879	0.318261	0.959641
3	0.031305	0.261218	0.977578

- Saving the model.

```
# Save final model + tokenizer
trainer.save_model("./news_distilbert_classification_model")
tokenizer.save_pretrained("./news_distilbert_classification_model")
print(f"\n Model saved to → {\"./news_distilbert_classification_model\"}")
```

Writing model shards: 100% 1/1 [00:00<00:00, 1.99it/s]

Model saved to → ./news_distilbert_classification_model

Step 9: Build inference pipeline

```
#Creating a pipeline for inferencing with our best_model_checkpoint, in this case checkpoint-189.
inference_pipeline = pipeline(task="text-classification",model = './news_distilbert_classification_model/checkpoint-189',batch_size=16,truncation=True)
```

Loading weights: 100%  104/104 [00:00<00:00, 662.29it/s, Materializing param=pre_classifier.weight]

```
#Calling the pipeline to perform inferencing on our test dataset.
list_ds = list(bbc_news_trans_ds_split['test']['text']) #Converting it to list as input is expected in format of list['str']
predictions_on_test_ds = inference_pipeline(list_ds)
```

```
predictions_on_test_ds[0:5] # We are getting a dictionary of keys 'label' and 'score'.
```

```
[{'label': 'LABEL_1', 'score': 0.9931897521018982},
 {'label': 'LABEL_2', 'score': 0.9925099611282349},
 {'label': 'LABEL_4', 'score': 0.9929726123809814},
 {'label': 'LABEL_1', 'score': 0.9948458671569824},
 {'label': 'LABEL_1', 'score': 0.994617998600061}]
```

Step 10: Transformer Predictions

- Run predictions on sample news articles

```
#Function to get only the label number e.g LABEL_1 => 1, LABEL_2 => 2 ; as we are expecting only a list of integers for different categories
y_pred = []
def get_label(predictions_on_test_ds):
    for prediction in predictions_on_test_ds:
        label = prediction['label'].split("_")[1]
        y_pred.append(int(label))
    return y_pred

y_pred = get_label(predictions_on_test_ds)
```

```
# We need the actual labels of test dataset so that we can calculate metrics such as accuracy score, confusion matrix
y_true = list(bbc_news_trans_ds_split['test']['label'])
```

Step 11: Model Evaluation

- Getting model accuracy and confusion matrix

```
print("Accuracy_score using transformer :",accuracy_score(y_true, y_pred))
```

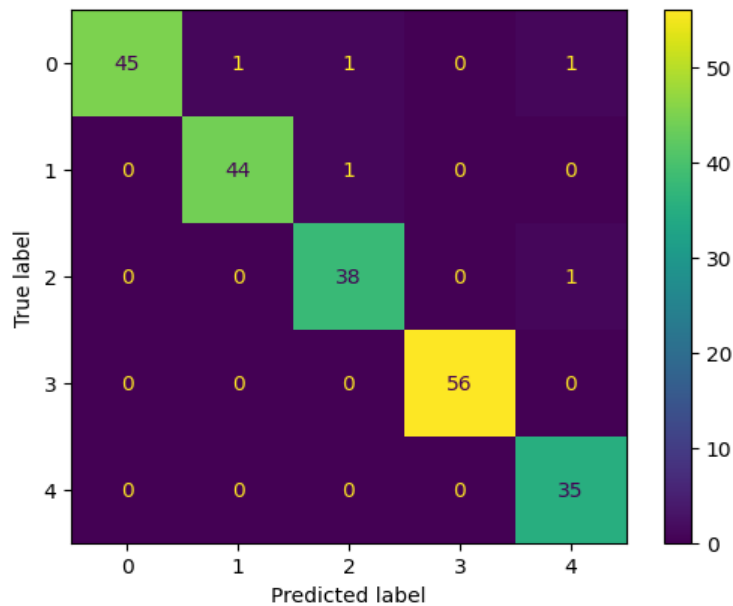
Accuracy_score using transformer : 0.9775784753363229

```
cm_transformer = confusion_matrix(y_true,y_pred)
print("Confusion Matrix \n",cm_transformer)
```

```
Confusion Matrix
[[45  1  1  0  1]
 [ 0 44  1  0  0]
 [ 0  0 38  0  1]
 [ 0  0  0 56  0]
 [ 0  0  0  0 35]]
```

```
# Creating a visually appealing confusion matrix
#Category mappings ==> 'business': 0, 'entertainment': 1, 'politics': 2, 'sport': 3, 'tech': 4
plt.figure(figsize=(8, 8))
cm_display = ConfusionMatrixDisplay(confusion_matrix=cm_transformer,
                                     display_labels=logistic_reg.classes_)
cm_display.plot()
plt.show()
```

<Figure size 800x800 with 0 Axes>



Step 12: Performance Comparison

- Compare ML vs Transformer predictions
- Why Transformers understand context better

Comparing ML vs Transformer predictions

Let's take a look at **accuracy score**:

Using Logistic regression model : **approx. 96%**

Using distilbert-base-uncased transformer model : **approx. 98%**

Although in our case the difference might not seem that vast, but the transformer model works on understanding the context of text rather than just the keywords. Therefore it is able to establish relation between words and predict the labels more accurately.

Phase 4: Generative AI – RAG (Retrieval-Augmented Generation)

Step 13: Document Creation

- Convert cleaned news articles into document format

- Each article acts as a knowledge source

```
bbc_news_og_ds['text'].to_csv('news_articles.txt', index=False, header=False)
```

+ Code

+ Markdown

```
# Loading the news_articles.txt as document
loader = TextLoader("news_articles.txt")
documents = loader.load()
```

```
# Chunking: Split text into manageable pieces
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = text_splitter.split_documents(documents)
```

Step 14: Embedding Generation and Vector Database

- Use the sentence transformer model: all-MiniLM-L6-v2
- Generate vector embeddings for articles
- Store embeddings in FAISS (Facebook AI Similarity Search)

```
# Embedding & Vector Store: Convert text to vectors and store them
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
vectorstore = FAISS.from_documents(chunks, embeddings)
```

Loading weights: 100%  103/103 [00:00<00:00, 734.96it/s, Materializing param=pooler.dense.weight]

```
# Generation: Use an LLM to answer using the retrieved context
# We use Qwen3 model with 0.6 billion parameters
qa_pipeline = pipeline("text-generation",
                        model="Qwen/Qwen3-0.6B",
                        device=0 if torch.cuda.is_available() else -1)
```

Loading weights: 100%  311/311 [00:00<00:00, 839.76it/s, Materializing param=model.norm.weight]

The tied weights mapping and config for this model specifies to tie model.embed_tokens.weight to lm_head.weight, but both are present. Update the config with `tie\_word\_embeddings=False` to silence this warning

+ Code

+ Markdown

We can also set the seed for transformer response reproducibility.

```
#Setting seed for response reproducibility
transformers.set_seed(42)
```

Step 15: RAG-Based Question Answering

RAG query pipeline code below

```

query_list = ['Summarize sports news article', 'What are the key points in this document?', 'Find top 3 news related to technology']
LLM_response = []
for query in query_list:
    docs = vectorstore.similarity_search(query, k=2) #Getting 2 relevant chunks for the query
    context = " ".join([document.page_content for document in docs]) #Combining the chunks to form context for the LLM
    prompt = f"Context: {context}\n\nQuestion: {query}\n\nAnswer:"
    result = qa_pipeline(prompt, max_new_tokens=100, do_sample=False)
    print(f"LLM Response: {result[0]['generated_text']}")
    print("-----")

```

RAG output:

LLM Response: Context: giving a greater depth to the growing wealth of analysis available during sports broadcasts. the technology has been created by bbc research and development for bbc broadcast and bbc outside broadcasts. bbc sport is the first client to start using the system. in order to keep audiences growing and growing... we need to work closely with people who create technology and innovation to bring sport to life said andrew thompson the bbc's head of development new media and sports news. we want sporting rivals go to extra time the current slew of sports games offers unparalleled opportunities for fans who like to emulate on-field action without ever moving from the couch. the two giants in the field - espn and ea sports - have been locked in a heavyweight battle for years. the latter is the world's largest games manufacturer. years of experience mean that the titles in their steady flow of sport sims are finely honed massively entertaining and ooze flair. sports broadcaster espn

Question: Summarize sports news article

Answer: The article discusses how the BBC Research and Development team has created a new technology system for sports broadcasts, which allows for greater depth in the analysis available during these broadcasts. This technology is used by BBC Sport as the first client to implement it. The goal is to keep audiences growing by working closely with people who create technology and innovation. The article also mentions that the BBC wants to provide extra time for fans to emulate on-field action without moving from the couch, and that the two major sports platforms, ESPN

LLM Response: Context: to transfer technology training capability and content from projects with foreign partners. we are defending our national sovereignty over the use of our national resources which must serve the endogenous development of the nation mr alvarez said. for this reason we are reviewing all memorandums of understanding all letters of intent all agreements that have been signed all contracts to check which of these comply with these directives. everything absolutely everything has to be propaganda exercise to try to sell the new eu to the country in advance of the forthcoming referendum and general election he asked. the tory spokesman also criticised the government for claiming the eu constitution would make europe easier to understand. the government last week had to publish a commentary of 500 pages to try and explain this easy and simple constitution to the british people he said. who are they trying to kid the proposed question for the constitution referendum

Question: What are the key points in this document?

Answer: The key points in this document are that the government is defending national sovereignty by using the use of national resources for endogenous development, and that the government is reviewing all memorandums of understanding, letters of intent, agreements, and contracts to check compliance with directives. everything has to be propaganda to try to sell the new eu to the country in advance of the referendum and general election. the government also claims the new eu constitution would make europe easier to understand, and that the government has published

LLM Response: Context: website which covers the impact of technology on media website which covers the impact of technology on media

Question: Find top 3 news related to technology

Answer: The top 3 news related to technology are: 1) The impact of technology on media is growing, 2) The rise of AI in media production, and 3) The role of social media in shaping public opinion.

Now, the answer is: 1) The impact of technology on media is growing, 2) The rise of AI in media production, and 3) The role of social media in shaping public opinion.

Final Analysis & Insights

Key Observations & Learnings

The key takeaways from this capstone project:

1. ML models are task specific and generally cannot handle complex task such as text generation very well.
2. Transformer model, if fine tuned on domain specific data can give accurate results for classifying labels as it tries to understand the relationship between the words and its similar meanings.
3. RAG-connects external knowledge source to LLM and help us generate responses which are up-to-date and relevant.